

Test Automation Proof of Architecture — 4-Week Plan

Three-Track POC: Traditional vs. AI-Assisted vs. AI-Native

Context: ~20 desktop/web/POS/Unix applications, ~1,500 existing manual test scenarios.
Goal: Fact-based, POC-backed recommendation of a test automation architecture. The POC runs the **same scenarios across three tracks** so the client sees, with measured data, where each level of the automation spectrum wins — and receives a tiered target-architecture recommendation plus a realistic coverage number and adoption roadmap.

The three tracks (definitions the whole engagement hangs on)

Track	Definition	Test artifact	Example stacks
A — Traditional	Deterministic scripted automation; human-authored locators & code; no AI anywhere	Script + locator repository	Selenium/Playwright, Robot Framework, WinAppDriver/FlaUI, SSHLibrary, UFT One (classic mode)
B — AI-Assisted	Architecture is still scripted & deterministic; AI helps humans build and maintain it — generative authoring, LLM conversion of manual scenarios, self-healing locators bolted onto scripts	Script (AI-generated or AI-repaired, human-owned and reviewed)	Playwright/Robot + LLM generation (Claude/Copilot), Healenium, Testim, mabl, Katalon AI, Tosca Copilot
C — AI-Native	AI is the execution engine; tests expressed as intent (natural language / model), interpreted against the UI at runtime; no locator repo, no conventional script; recognition and healing are intrinsic	Intent / NL test definition	testRigor, Functionize, agentic modes of Eggplant / KaneAI, computer-use agents, vision-based runtime recognition

Classification of common capabilities (make this explicit in the client report):

Capability	Track
LLM converts written manual scenarios into runnable scripts	B (AI-assisted)
Copilot/NL authoring inside a scripted framework	B
Self-healing locators on scripted tests	B
Vision-AI runtime object recognition (no locators)	C (AI-native)
Agentic execution from pure natural-language intent	C
Visual AI validation (layout/rendering assertions)	Borderline — AI-native validation usable inside B frameworks
AI test-data generation, flaky-test triage	Track-agnostic accelerators

Guiding principles

- Evaluate against *this* client's estate, not in the abstract; POC apps must include the hardest 20% (POS with peripherals, legacy desktop, terminal apps).
- Same scenarios across all three tracks — every claim becomes a measured delta.
- Working hypothesis to test (state it openly): **B is the workhorse for regression at scale; C is deployed surgically** where traditional recognition fails or scripting isn't worth it. The POC either confirms or overturns this with data.
- Measure honestly: alongside success rates, capture false positives, run-to-run determinism, and per-run cost.
- Present coverage as a fact-based number with buckets, not a promise of 100%.

What each track must prove (POC hypotheses & metrics)

#	Hypothesis	Metric	Track
1	LLM generation cuts authoring effort 40–70% on the 1,500-scenario backlog	Authoring time per scenario B vs. A; % of generated scripts runnable with < 15 min fix-up	B vs. A
2	Self-healing cuts maintenance effort 50–80% after UI change	Tests broken, healing success rate, human fix time after deliberate UI change	B vs. A
3	AI-native vision/agent execution unlocks apps traditional tools cannot see	% of "hard" POS/terminal/legacy scenarios automatable only in Track C	C vs. A/B
4	AI-native intent-based tests are reliable enough for regression	Pass-rate consistency over 20 runs; variance in steps taken; false pass/fail rate	C
5	AI-native maintenance is near-zero when UI changes	Same UI-change experiment run against Track C	C vs. B
6	Per-run economics differ materially	Cost per scenario run (licenses + tokens/compute) per track at regression scale (1,500 × release cadence)	A/B/C
7	All tracks are CI/CD- and audit-compatible	Pipeline integration success; auditability of healing events and agent decisions	A/B/C

Known risks to probe, especially on Track C: hallucinated/weak assertions, non-determinism across runs, audit/compliance acceptability, data privacy when screens go to cloud AI (PCI concerns on POS), token-based cost at regression volume, vendor lock-in.

Week 1 — Discovery & Estate Analysis

Day 1 — Kickoff & access

- Kickoff: scope, success criteria, decision-makers; present the three-track methodology and get explicit buy-in for it.
- Request access: apps, environments, test management (Jira/ALM/Xray), CI/CD, source control.

- **AI governance question on day one:** is cloud AI permitted? PCI / payment-data constraints on POS screens? Data residency? Self-hosted LLM required? The answer shapes which Track B/C tools are even eligible.

Day 2 — Application inventory

- Landscape matrix for all ~20 apps: type, technology, version, hosting, release cadence, criticality, cross-app E2E flows.
- Flag apps where traditional object recognition will likely fail — prime Track C candidates.

Day 3 — Test scenario analysis

- Export ~1,500 scenarios; classify by app, type, frequency, data/environment dependency, manual-only constraints (hardware, printers, card readers).
- Tag: *automatable now / with effort / not automatable*.
- Assess documentation quality: % of backlog that is "LLM-convertible as written" (Track B fuel) vs. needs rewriting.

Day 4 — Stakeholder interviews & constraints

- QA/dev/ops interviews: pain points, past automation failures, skill profile (coders vs. non-coders — relevant to Track C's NL authoring), budget appetite.
- Probe the top maintenance pain (self-healing story) and audit/compliance expectations (Track C acceptability).

Day 5 — Evaluation framework & tool longlist

- Weighted criteria (example): technology coverage 20%, authoring effort 15%, maintainability incl. self-healing 15%, AI maturity & accuracy 10%, determinism & auditability 8%, CI/CD & reporting 8%, skill fit 8%, licensing/TCO incl. AI consumption 10%, vendor support & roadmap 6%.
 - Longlist mapped to tracks:
 - **Track A candidates:** Playwright/Selenium, Robot Framework + FlaUI/WinAppDriver/SSHLibrary, UFT One, Ranorex, TestComplete.
 - **Track B candidates:** Track A stack + LLM generation (Claude/Copilot) + Healenium; or Testim, mabl, Katalon AI, Tosca with Copilot/Vision assist.
 - **Track C candidates:** testRigor, Functionize, Eggplant/KaneAI agentic modes, computer-use agents; AppliTools as visual-AI layer.
 - Client sign-off on criteria, weights, and three-track methodology **before** the POC.
-

Week 2 — Shortlisting & POC Setup

Day 6 — Shortlist & POC scope

- Select **one stack per track** (Track B is often Track A + AI layers — deliberately, since that mirrors a realistic adoption path).
- Select 3-4 representative apps: modern web, legacy desktop, POS, Unix/terminal.
- Select 10-12 common scenarios to run in **all three tracks**, plus 3-4 "hard" scenarios reserved to test Track C's unlock claim (hypothesis 3).

Day 7 — Environment & licenses

- Trials/installs; provision POC machines/VMs; verify each stack can see each app type (object spy / vision check) — eliminates weak candidates immediately.
- Enable AI features in trials (often vendor-gated); set up LLM API access within client security rules; masked/synthetic data policy for anything sent to cloud AI.

Day 8 — POC design & scorecard

- Lock the scorecard: per track per scenario — authoring time, stability over N runs, recognition quality, debugging experience, run cost; plus Track-B healing metrics and Track-C determinism metrics (hypotheses table above).
- Framework skeletons: Track A/B page-object structure, data handling, reporting; Track C project/workspace setup.

Day 9 — Build: Track A (traditional baseline)

- Automate the common scenarios traditionally. Log effort meticulously — this is the denominator for every improvement claim.

Day 10 — Track A finish + Track B conversion pilot + checkpoint

- Finish Track A; run 10+ times for stability baseline.
 - **Track B pilot:** feed 20-30 real client manual scenarios into LLM generation targeting the Track A framework; measure % runnable with < 15 min fix-up.
 - Weekly checkpoint: estate analysis, Track A baseline numbers, first conversion-rate data. Use this meeting to reset any "100% automation" expectations with facts.
-

Days 11-12 — Build: Track B (AI-assisted)

- Complete the same scenario set via AI-assisted authoring (LLM generation + copilot) into the scripted framework; wire in self-healing (e.g., Healenium or platform-native).
- Capture authoring-time deltas vs. Track A per scenario; log human review effort on generated code (that's a real cost — count it).

Day 13 — Build: Track C (AI-native)

- Express the same scenarios as natural-language intent in the AI-native platform; run the reserved "hard" scenarios (POS/terminal/legacy) that Tracks A/B struggled with.
- Run each Track C scenario 10-20 times **today**, not just once — run-to-run determinism is the make-or-break metric, and agentic tools often shine on run 1 and wobble on run 7.

Day 14 — CI/CD & operability, all tracks

- Integrate all three tracks with the client's pipeline (Jenkins/Azure DevOps/GitLab): triggered runs, parallelism, reports, results back to Jira/ALM.
- Auditability check: are Track B healing events logged and reviewable? Can Track C explain what an agent did and why? Compliance stakeholders will ask.

Day 15 — The maintenance experiment (the money slide)

- Deliberately change UI elements/locators/screens in the POC apps. Measure per track: tests broken, self-healing success rate (B), agent adaptation (C), human fix time (A/B/C), false-heal rate.
- Re-run all suites 20+ times; final flakiness and false-positive/negative numbers.
- Collect licensing quotes incl. AI consumption pricing; model cost per regression cycle per track at 1,500-scenario scale.

Week 4 — Analysis, Recommendation & Roadmap

Day 16 — Consolidate POC data

- Fill the weighted scorecard per track with measured data only.
- Extrapolate backlog conversion effort: e.g., "1,280 automatable scenarios $\approx$ 14 person-months via Track A, $\approx$ 6 via Track B generation, $\approx$ 4 via Track C NL authoring — at these respective reliability and run-cost levels."

Day 17 — Coverage & ROI model

- Coverage statement in five buckets: automatable via Track A/B / **automatable only via Track C** (the AI-native unlock — your headline) / needs app or testability changes / remains manual (hardware, exploratory judgment).
- ROI per track and for the tiered combination: authoring savings + maintenance savings – AI licensing/consumption costs; break-even timeline; sensitivity to release cadence.

Day 18 — Tiered recommendation & roadmap

- Recommendation traced to evidence — expected shape (confirm or overturn with your data): **Track B as the regression workhorse** (deterministic, auditable, CI-friendly, cheap per run) + **Track C surgically** for the apps/scenarios only it can reach and for low-frequency cases where scripting isn't worth it; Track A retained only where AI is prohibited by governance.
- Roadmap: framework build-out; AI-assisted mass conversion of the backlog with human review gates; Track C pilot on the POS/terminal apps; team training (prompting and reviewing AI-generated tests is a new skill); governance — review gates for AI-generated assertions, self-healing audit policy, agentic-run approval policy, data-masking for AI services; 6/12/18-month milestones.

Day 19 — Report & presentation prep

- POA report: estate analysis, three-track methodology, measured results per hypothesis, risks, tiered recommendation, cost model, roadmap.
- Executive deck (10–12 slides). Lead with three numbers: authoring-time reduction (B vs. A), maintenance-effort reduction (B/C vs. A), coverage unlocked only by AI-native (C). Technical appendix behind it.
- Dry run / peer review.

Day 20 — Presentation & handover

- Present exec summary first; evidence on demand. If possible, one live moment: break a locator on stage — Track A fails, Track B heals, Track C doesn't care.
 - Hand over POC assets: code, intent definitions, scorecards, raw data, AI-generated-script samples with review notes.
 - Agree next steps: procurement, pilot team, conversion project kickoff.
-

1. Application landscape & technology matrix (with Track C suitability flags)
2. Scenario classification + LLM-convertibility rate
3. Client-approved weighted evaluation framework and three-track methodology
4. POC assets + head-to-head scorecard: Track A vs. B vs. C, per hypothesis
5. Five-bucket coverage statement incl. "unlocked only by AI-native"
6. 3-year TCO & ROI model per track and for the tiered combination
7. Tiered recommendation report + executive presentation
8. Adoption roadmap incl. AI governance and backlog-conversion plan

Key risks to manage

- **Access/license delays** — raise Day 1, escalate Day 3; AI features often need vendor enablement in trials.
- **Cherry-picked scenarios** — the hardest app types must be in scope; Track C claims must survive POS and terminal apps.
- **AI over-claiming** — report false-positive and determinism numbers alongside success rates; count human review time on generated tests as real cost.
- **Data privacy** — never send real payment/customer data to cloud AI; masked/synthetic data from Day 7; PCI check for POS screens.
- **"100% automation" expectation** — reset with data at the Week 2 checkpoint; AI raises the ceiling but hardware and exploratory judgment keep some scenarios manual.
- **Tool/track bias** — pre-agreed weights and identical scenarios across tracks are your defense.